

Bolco de estruturação/Descodificação V1.0

Date: 25/01/2026

Autor: Luís Costa

Indices

1.	Diagrama do Bloco de Memória (16 Bytes)	2
2.	Implementação em C/C++ (Struct)	3
3.	Funções de Codificação (Packing)	3
4.	Implementação da Encriptação (AES-128)	4
4.1.	Chave de Segurança	4
4.2.	Funções de Cifra e Decifra	4
5.	Código do Emissor (Firmware do Colete)	5
6.	Código do Recetor (Tablet / Dashboard)	8

1. Diagrama do Bloco de Memória (16 Bytes)

A tabela que se segue representa como os bits estão organizados na memória antes da encriptação AES-128.

		Bits								Descrição
		7	6	5	4	3	2	1	0	
Bytes	00	Vest ID (int8_t)								ID único
	01	FLG	SpO2 (7 bits)							Flag Alerta + SpO2
	02	Heart Rate (8 bits) Padding (7 bits) / 9ºbit HR (bit 0)								HR (9 bits total)
	03									HR MSB + Padding
	04	Temperature (int16_t)								Temp * 100
	05									
	06 ... 09	Latitude (int32_t)								Lat *10^7
	10 ... 13	Longitude (int32_t)								Lon *10^7
	14	Altitude (int16_t)								Altitude em Metros
	15									

2. Implementação em C/C++ (Struct)

Para garantir que o compilador não adiciona "padding" (espaços vazios) entre as variáveis, é obrigatório o uso do atributo packed.

```
typedef struct __attribute__((packed)) {  
    int8_t vestID;  
    uint8_t spo2_header;  
    uint16_t heartRate;  
    int16_t temperature;  
    int32_t latitude;  
    int32_t longitude;  
    int16_t altitude;  
} ResQSensePacket;
```

3. Funções de Codificação (Packing)

Como os campos spo2_header e heartRate misturam dados e flags no mesmo byte, não se deve escrever neles diretamente. Utilizamos funções auxiliares para garantir que os bits ficam na posição correta.

Notas Importantes de Conversão:

Temperatura: 36.50°C deve ser enviado como 3650.

Coordenadas: 40.6412° deve ser enviado como 406412000.

Heart Rate: Suporta até 511 BPM (9 bits), cobrindo casos de taquicardia extrema.

```
ResQSensePacket createPacket(int id, bool alertFlag, int spo2, int hr, float temp, float lat, float lon, int alt) {  
    ResQSensePacket packet;  
  
    packet.vestID = (int8_t)id;  
    uint8_t flagBit = alertFlag ? 1 : 0;  
    packet.spo2_header = (flagBit << 7) | (spo2 & 0x7F);  
    packet.heartRate = (uint16_t)(hr & 0x01FF);  
    packet.temperature = (int16_t)(temp * 100.0f);  
    packet.latitude = (int32_t)(lat * 10000000.0f);  
    packet.longitude = (int32_t)(lon * 10000000.0f);  
}
```

```

packet.altitude = (int16_t)alt;

return packet;
}

```

4. Implementação da Encriptação (AES-128)

Para garantir a confidencialidade dos dados transmitidos via LoRa, utiliza-se o algoritmo **AES-128**. Este modo foi escolhido por permitir encriptar o bloco de 16 bytes da estrutura `ResQSensePacket` num bloco cifrado de exatamente 16 bytes, sem aumentar o tamanho do payload e mantendo a eficiência energética.

O ESP32 possui aceleração de hardware para criptografia através da biblioteca nativa [mbedtls](#).

4.1. Chave de Segurança

A chave tem 128 bits (16 bytes) e tem de ser rigorosamente igual no código do Emissor e do Recetor.

```

const unsigned char AES_KEY[16] = {
    0x2B, 0x7E, 0x15, 0x16, 0x28, 0xAE, 0xD2, 0xA6,
    0xAB, 0xF7, 0x15, 0x88, 0x09, 0xCF, 0x4F, 0x3C
};

```

4.2. Funções de Cifra e Decifra

Estas funções utilizam o contexto da biblioteca `mbedtls` para transformar a estrutura de dados num buffer encriptado e vice-versa.

Função para Encriptar (No Emissor)

```

// Entrada: Struct organizada | Saída: Buffer de 16 bytes ilegíveis
void encryptPacket(const ResQSensePacket& inputPacket, uint8_t* outputBuffer) {
    mbedtls_aes_context aes;

    mbedtls_aes_init(&aes);

```

```

mbedtls_aes_setkey_enc(&aes, AES_KEY, 128);

// O cast (unsigned char*) trata a struct como um array de bytes crus
mbedtls_aes_crypt_ecb(&aes, MBEDTLS_AES_ENCRYPT, (const unsigned char*)&inputPacket,
outputBuffer);

mbedtls_aes_free(&aes);
}

```

Função para Desencriptar (No Recetor)

// Entrada: Buffer encriptado | Saída: Struct organizada

```

void decryptPacket(const uint8_t* inputBuffer, ResQSensePacket* outputPacket) {
    mbedtls_aes_context aes;

    mbedtls_aes_init(&aes);
    mbedtls_aes_setkey_dec(&aes, AES_KEY, 128);

    mbedtls_aes_crypt_ecb(&aes, MBEDTLS_AES_DECRYPT, inputBuffer, (unsigned char*)outputPacket);

    mbedtls_aes_free(&aes);
}

```

5. Código do Emissor (Firmware do Colete)

O código do emissor integra a leitura (simulada) dos sensores, o empacotamento na estrutura definida e a encriptação antes do envio via LoRa.

```

#include <SPI.h>
#include <LoRa.h>
#include "mbedtls/aes.h"

// --- DEFINIÇÕES DE HARDWARE ---
#define SCK    5
#define MISO   19
#define MOSI   27
#define SS     18
#define RST    14

```

```

#define DIO0 26

// --- CONFIGURAÇÃO LORA ---
#define BAND 868E6 // 868 MHz

// --- ESTRUTURAS E CHAVES ---
const unsigned char AES_KEY[16] = {
    0x2B, 0x7E, 0x15, 0x16, 0x28, 0xAE, 0xD2, 0xA6,
    0xAB, 0xF7, 0x15, 0x88, 0x09, 0xCF, 0x4F, 0x3C
};

typedef struct __attribute__((packed)) {
    int8_t vestID;
    uint8_t spo2_header;
    uint16_t heartRate;
    int16_t temperature;
    int32_t latitude;
    int32_t longitude;
    int16_t altitude;
} ResQSensePacket;

// Buffer para guardar os dados encriptados
uint8_t encryptedBuffer[16];

// --- FUNÇÃO DE PACKING ---
ResQSensePacket createPacket(int id, bool alertFlag, int spo2, int hr, float temp, float lat, float lon, int alt) {
    ResQSensePacket packet;
    packet.vestID = (int8_t)id;
    uint8_t flagBit = alertFlag ? 1 : 0;
    packet.spo2_header = (flagBit << 7) | (spo2 & 0x7F);
    packet.heartRate = (uint16_t)(hr & 0x01FF);
    packet.temperature = (int16_t)(temp * 100.0f);
    packet.latitude = (int32_t)(lat * 10000000.0f);
    packet.longitude = (int32_t)(lon * 10000000.0f);
    packet.altitude = (int16_t)alt;
    return packet;
}

// --- FUNÇÃO DE ENCRIPTAÇÃO ---
void encryptPacket(const ResQSensePacket& inputPacket, uint8_t* outputBuffer) {

```

```

    mbedtls_aes_context aes;
    mbedtls_aes_init(&aes);
    mbedtls_aes_setkey_enc(&aes, AES_KEY, 128);
    mbedtls_aes_crypt_ecb(&aes, MBEDTLS_AES_ENCRYPT, (const unsigned char*)&inputPacket,
outputBuffer);
    mbedtls_aes_free(&aes);
}

void setup() {
    Serial.begin(115200);

    SPI.begin(SCK, MISO, MOSI, SS);
    LoRa.setPins(SS, RST, DIO0);

    if (!LoRa.begin(BAND)) {
        Serial.println("Erro ao iniciar LoRa!");
        while (1);
    }
    Serial.println("ResQSense TX Iniciado (AES-128 Ativo).");
}

void loop() {
    // Simulação de Leitura de Sensores
    int myID = 1;
    bool panicBtn = false;
    int valSpO2 = 98;
    int valHR = 72;
    float valTemp = 36.65;
    float valLat = 40.64427;
    float valLon = -8.64554;
    int valAlt = 15;

    // Criar Pacote (Packing)
    ResQSensePacket packet = createPacket(myID, panicBtn, valSpO2, valHR, valTemp, valLat, valLon, valAlt);

    // Encriptar
    encryptPacket(packet, encryptedBuffer);

    // Enviar via LoRa
    LoRa.beginPacket();

```

```

    LoRa.write(encryptedBuffer, 16); // Envia exatamente 16 bytes
    LoRa.endPacket();

    Serial.println("Pacote encriptado enviado.");

    // Aguardar 10 segundos
    delay(10000);
}

```

6. Código do Recetor (Tablet / Dashboard)

O código do recetor, recebe o pacote encriptado, descripta-o utilizando a mesma chave e reconverte os bits (faz unpacking) para valores legíveis.

```

#include <SPI.h>
#include <LoRa.h>
#include "mbedtls/aes.h"

// --- DEFINIÇÕES DE HARDWARE ---
#define SCK    5
#define MISO   19
#define MOSI   27
#define SS     18
#define RST    14
#define DIO0    26
#define BAND   868E6

// --- CHAVE (IGUAL AO EMISSOR) ---
const unsigned char AES_KEY[16] = {
    0x2B, 0x7E, 0x15, 0x16, 0x28, 0xAE, 0xD2, 0xA6,
    0xAB, 0xF7, 0x15, 0x88, 0x09, 0xCF, 0x4F, 0x3C
};

// --- ESTRUTURAS ---
typedef struct __attribute__((packed)) {
    int8_t vestID;
    uint8_t spo2_header;
    uint16_t heartRate;
}

```



```

    int16_t temperature;
    int32_t latitude;
    int32_t longitude;
    int16_t altitude;
} ResQSensePacket;

// Estrutura Legível para o Dashboard
typedef struct {
    int vestID;
    bool isAlerting;
    int spo2;
    int heartRate;
    float temperature;
    float latitude;
    float longitude;
    int altitude;
} DashboardData;

// --- FUNÇÃO DE DESENCRIPTAÇÃO ---
void decryptPacket(const uint8_t* inputBuffer, ResQSensePacket* outputPacket) {
    mbedtls_aes_context aes;
    mbedtls_aes_init(&aes);
    mbedtls_aes_setkey_dec(&aes, AES_KEY, 128);
    mbedtls_aes_crypt_ecb(&aes, MBEDTLS_AES_DECRYPT, inputBuffer, (unsigned char*)outputPacket);
    mbedtls_aes_free(&aes);
}

// --- FUNÇÃO DE DECODIFICAÇÃO (UNPACKING) ---
DashboardData unpackPacket(const ResQSensePacket& packet) {
    DashboardData data;

    data.vestID = packet.vestID;

    // Extrair bits: Flag (bit 7) e SpO2 (bits 0-6)
    data.isAlerting = (packet.spo2_header >> 7) & 0x01;
    data.spo2 = packet.spo2_header & 0x7F;

    // Extrair HR (Máscara 0x1FF para pegar os 9 bits)
    data.heartRate = packet.heartRate & 0x01FF;

```

```

// Converter valores escalados de volta para float
data.temperature = (float)packet.temperature / 100.0f;
data.latitude = (float)packet.latitude / 10000000.0f;
data.longitude = (float)packet.longitude / 10000000.0f;

data.altitude = packet.altitude;

return data;
}

void setup() {
    Serial.begin(115200);
    SPI.begin(SCK, MISO, MOSI, SS);
    LoRa.setPins(SS, RST, DIO0);

    if (!LoRa.begin(BAND)) {
        Serial.println("Erro LoRa RX");
        while (1);
    }
    Serial.println("ResQSense RX Iniciado (A aguardar dados)...");
}

void loop() {
    int packetSize = LoRa.parsePacket();

    // Verificação de segurança: O pacote tem de ter exatamente 16 bytes
    if (packetSize == sizeof(ResQSensePacket)) {

        uint8_t receivedBuffer[16];
        ResQSensePacket decryptedData;

        // Ler dados encriptados
        LoRa.readBytes(receivedBuffer, 16);

        // Descriptar
        decryptPacket(receivedBuffer, &decryptedData);

        // Decodificar para formato legível
        DashboardData view = unpackPacket(decryptedData);
    }
}

```

```

// Output para Serial/Interface
Serial.print("ID: "); Serial.print(view.vestID);
Serial.print(" | Alerta: "); Serial.print(view.isAlerting ? "SIM" : "Nao");
Serial.print(" | BPM: "); Serial.print(view.heartRate);
Serial.print(" | SpO2: "); Serial.print(view.spo2);
Serial.print(" | Temp: "); Serial.print(view.temperature);
Serial.print(" | GPS: "); Serial.print(view.latitude, 7);
Serial.print(", "); Serial.println(view.longitude, 7);

} else if (packetSize > 0) {
    Serial.print("Pacote ignorado (tamanho incorreto): ");
    Serial.println(packetSize);
}
}

```

Proximos passos

Testar com duas ESP32 os códigos para testar valores de Time on Air e SpreadFactor e confirmar que o recetor recebe tudo sem perdas mesmo em situações adversas, de forma a avançar para a leitura dos valores nos sensores e passagem para o dashboard

Bibliografia

Encriptação:

<https://github.com/alvingigo/AES-128/blob/main/aes128.cpp> (Chave de segurança)

<https://mbed-tls.readthedocs.io/en/latest/kb/how-to/encrypt-with-aes-cbc/> (mbedtls)

Ferramenta de Validação AES (Para testes):

<https://gchq.github.io/CyberChef/>(CyberChef - Ótimo para confirmar se a encriptação está correta manualmente)